

tidyr Package in R

tidyr is used to **clean and reshape data**.

Main goal:

Convert messy data into tidy format

What is Tidy Data?

A dataset is **tidy** when:

- Each column = one variable
- Each row = one observation
- Each value = one cell

Example (Untidy Data)

```
data <- data.frame(  
  gene = c("BRCA1", "TP53"),  
  control = c(5, 8),  
  treated = c(12, 15)  
)
```

Problem:

- Control & Treated are separate columns (not tidy)

1. pivot_longer() (MOST IMPORTANT)

Convert **wide** → **long format**

```
library(tidyr)  
  
long_data <- data %>%  
  pivot_longer(cols = control:treated,  
               names_to = "condition",  
               values_to = "expression")
```

Output:

gene	condition	expression
BRCA1	control	5
BRCA1	treated	12

2. pivot_wider()

Convert **long** → **wide format**

```
wide_data <- long_data %>%  
  pivot_wider(names_from = condition,  
              values_from = expression)
```

3. separate()

Split one column into multiple columns

```
data <- data.frame(info = c("BRCA1_Control", "TP53_Treated"))  
  
data %>%  
  separate(info, into = c("gene", "condition"), sep = "_")
```

4. unite()

Combine multiple columns into one

```
data %>%  
  unite("combined", gene, condition, sep = "_")
```

5. drop_na()

Remove missing values

```
data %>% drop_na()
```

6. replace_na()

Replace missing values

```
data %>% replace_na(list(expression = 0))
```

7. fill()

Fill missing values (forward/backward)

```
data %>% fill(gene)
```

8. complete()

Add missing combinations

```
data %>% complete(gene, condition)
```

File Handling in R

File handling means **reading data from files** and **writing data to files** in R.

Why File Handling is Important?

- Import experimental data (CSV, Excel)
- Save analysis results
- Share data with others
- Use datasets in research

Types of Files Supported in R

- CSV (Comma Separated Values)
- Excel (.xlsx)
- Text files (.txt)
- R-specific files (.RData, .rds)

1. Reading Files in R

(A) Read CSV File

```
data <- read.csv("data.csv")
```

Most commonly used in biology and research

(B) Read Excel File

Use package: readxl

```
library(readxl)
data <- read_excel("data.xlsx")
```

(C) Read Text File

```
data <- read.table("data.txt", header = TRUE)
```

(D) Read R Data File

```
load("data.RData")
```

2. Writing Files in R

(A) Write CSV File

```
write.csv(data, "output.csv", row.names = FALSE)
```

(B) Write Text File

```
write.table(data, "output.txt", row.names = FALSE)
```

(C) Save R Data

```
save(data, file = "data.RData")
```

(D) Save Single Object (.rds)

```
saveRDS(data, "data.rds")
```

Load later:

```
data <- readRDS("data.rds")
```

3. Working Directory (IMPORTANT)

```
getwd()          # Check current directory
setwd("D:/R_Files") # Set new directory
```

4. Checking File Structure

```
head(data)      # First rows
str(data)       # Structure
summary(data)   # Summary stats
```

5. Important Arguments

Argument	Meaning
header = TRUE	First row is column name
sep = ","	Separator
row.names = FALSE	Do not write row numbers

Tidyverse in R

Tidyverse is a **collection of R packages** used for:

- Data manipulation
- Data visualization
- Data cleaning

It makes R code:

- Cleaner
- Faster
- Easier to read

Tidyverse is developed by:

- Hadley Wickham

It includes important packages:

Package	Use
dplyr	Data manipulation
ggplot2	Data visualization
tidyr	Data cleaning
readr	Fast file reading
tibble	Modern data frames

Package	Use
stringr	String handling
forcats	Factor handling

Install & Load Tidyverse

```
install.packages("tidyverse")
library(tidyverse)
```

Loads all major packages together

Tidy Data Concept (Very Important)

Data should be in **tidy format**:

1. Each column = variable
2. Each row = observation
3. Each value = single data

Untidy Data Example

Gene	Control	Treated
BRCA1	5	12

Tidy Data Format

Gene	Condition	Expression
BRCA1	Control	5
BRCA1	Treated	12

This format is required for analysis

1. dplyr (Data Manipulation)

Already covered but used inside tidyverse

```
data %>%
  filter(condition == "Treated") %>%
  group_by(gene) %>%
  summarise(mean_exp = mean(expression))
```

2. ggplot2 (Visualization)

```
ggplot(data, aes(x=gene, y=expression)) +
  geom_bar(stat="identity")
```

3. readr (Fast File Reading)

Better than `read.csv()`

```
data <- read_csv("gene_data.csv")
```

Advantages:

- Faster
- Cleaner output

4. tibble (Modern Data Frame)

Improved version of data frame

```
data <- tibble(  
  gene = c("BRCA1", "TP53"),  
  expression = c(10, 15)  
)  
  
print(data)
```

Differences:

- Better display
- No automatic conversion

5. tidyr (Data Cleaning)

Used to reshape data

Example: Convert Wide → Long Format

```
library(tidyr)  
  
data <- data.frame(  
  gene = c("BRCA1"),  
  Control = 5,  
  Treated = 12  
)  
  
long_data <- pivot_longer(data,  
                           cols = c(Control, Treated),  
                           names_to = "condition",  
                           values_to = "expression")  
  
print(long_data)
```

Very important for **bioinformatics**

6. stringr (String Handling)

```
library(stringr)  
  
str_length("Neeraj")  
str_to_upper("gene")
```

7. Pipe Operator %>% (Very Important)

Passes output to next function

```
data %>%  
  filter(expression > 10) %>%  
  arrange(desc(expression))
```

Makes code:

- Clean
- Easy to understand

Real Biotech Workflow Using Tidyverse

```
library(tidyverse)  
  
# Step 1: Read data  
data <- read_csv("gene_data.csv")  
  
# Step 2: Clean data  
data <- data %>%  
  filter(!is.na(expression))  
  
# Step 3: Analysis  
result <- data %>%  
  group_by(gene, condition) %>%  
  summarise(mean_exp = mean(expression))  
  
# Step 4: Visualization  
ggplot(result, aes(x=gene, y=mean_exp, fill=condition)) +  
  geom_bar(stat="identity")  
  
# Step 5: Save result  
write_csv(result, "final_output.csv")
```

Advantages of Tidyverse

- Easy syntax
- Powerful pipelines
- Industry standard
- Used in:
 - Data Science
 - Bioinformatics
 - Research

Summary

- Tidyverse = collection of powerful packages
- Most important:
 - dplyr
 - ggplot2
 - tidyr
- Use %>% for clean workflow
- Follow **tidy data format**

Bioconductor in R

Bioconductor is a **specialized project in R** for:

- Genomics
- Bioinformatics
- Biological data analysis

It provides **hundreds of packages** for analyzing:

- Gene expression
- RNA-seq data
- DNA sequences
- Microarray data

1. DESeq2 – RNA-Seq Analysis

DESeq2

- Find **differentially expressed genes**
- Compare Control vs Treated

Example Code:

```
library(DESeq2)

# Sample count data
countData <- matrix(c(10,20,30,15,25,35), nrow=3)
colData <- data.frame(condition = c("Control","Treated"))

dds <- DESeqDataSetFromMatrix(countData = countData,
                              colData = colData,
                              design = ~ condition)

dds <- DESeq(dds)

results(dds)
```

Output: Genes that are up/down regulated

2. edgeR – Gene Expression Analysis

edgeR

- Differential expression (similar to DESeq2)
- Works well for small sample sizes

Example Code:

```
library(edgeR)

counts <- matrix(c(10,20,30,15,25,35), nrow=3)
group <- factor(c("Control","Treated"))

dge <- DGEList(counts=counts, group=group)
```



```
dge <- calcNormFactors(dge)
dge <- estimateDisp(dge)
et <- exactTest(dge)
topTags(et)
```

Output: Significant genes

3. limma – Microarray Analysis

limma

- Microarray data analysis
- Also used for RNA-seq (with voom)

Example Code:

```
library(limma)

# Expression matrix
expr <- matrix(rnorm(100), nrow=10)

group <- factor(c("Control", "Treated"))

design <- model.matrix(~group)

fit <- lmFit(expr, design)
fit <- eBayes(fit)

topTable(fit)
```

Output: Top differentially expressed genes

4. GenomicRanges – Genome Data

GenomicRanges

- Handle genomic regions (chromosome positions)
- Overlap analysis

Example Code:

```
library(GenomicRanges)

gr <- GRanges(seqnames = "chr1",
              ranges = IRanges(start = c(100, 200),
                              end = c(150, 250)))

gr
```

Output: Genomic regions object

5. Biostrings – DNA/RNA Sequences

Biostrings

- DNA/RNA sequence analysis
- Pattern matching

Example Code:

```
library(Biostrings)

dna <- DNAString("ATGCGTACGTTAG")

# Count nucleotides
alphabetFrequency(dna)

# Find pattern
matchPattern("CGT", dna)
```

Output:

- Base count (A, T, G, C)
- Pattern positions

Summary Table

Package	Use
DESeq2	RNA-seq differential expression
edgeR	Gene expression analysis
limma	Microarray + RNA-seq
GenomicRanges	Genome intervals
Biostrings	DNA/RNA sequence

Project: RNA-Seq Style Gene Expression Analysis Pipeline

Objective

Complete pipeline banana:

- Data cleaning
- Transformation (log, fold change)
- Statistical summary
- Visualization
- Biological interpretation

Step 1: Create Dataset (Simulated RNA-Seq)

```
data <- data.frame(
  gene = rep(c("BRCA1", "TP53", "EGFR", "MYC"), each = 4),
  condition = rep(c("Control", "Control", "Treated", "Treated"), times = 4),
  expression = c(5, 6, 12, 13,
                 8, 9, 15, 16,
                 10, 11, 18, 19,
                 7, 8, 20, 22))
```

```
)  
  
print(data)
```

Step 2: Load Libraries

```
library(dplyr)  
library(tidyr)  
library(ggplot2)
```

Step 3: Data Cleaning & Check

```
head(data)  
summary(data)
```

Step 4: Calculate Mean Expression

```
summary_data <- data %>%  
  group_by(gene, condition) %>%  
  summarise(mean_exp = mean(expression),  
            sd_exp = sd(expression))  
  
print(summary_data)
```

Step 5: Reshape Data (Wide Format)

```
wide_data <- summary_data %>%  
  pivot_wider(names_from = condition, values_from = mean_exp)  
  
print(wide_data)
```

Step 6: Calculate Fold Change + LogFC 🧠

```
analysis_data <- wide_data %>%  
  mutate(  
    fold_change = Treated / Control,  
    logFC = log2(fold_change)  
  )  
  
print(analysis_data)
```

Step 7: Identify Significant Genes

```
significant_genes <- analysis_data %>%  
  filter(fold_change > 1.5)  
  
print(significant_genes)
```

Step 8: Visualization (Bar Plot)

```
ggplot(summary_data, aes(x = gene, y = mean_exp, fill = condition)) +  
  geom_bar(stat = "identity", position = "dodge") +  
  ggtitle("Gene Expression Comparison") +  
  ylab("Mean Expression") +  
  theme_minimal()
```

Step 9: Log Fold Change Plot (IMPORTANT 🧠)

```
ggplot(analysis_data, aes(x = gene, y = logFC, fill = gene)) +  
  geom_bar(stat = "identity") +
```

```
ggtitle("Log Fold Change of Genes") +  
ylab("log2 Fold Change") +  
theme_minimal()
```

Step 10: Biological Interpretation

From results:

- All genes show **higher expression in Treated**
- $\log FC > 0 \rightarrow$ **upregulated genes**
- MYC shows highest increase $\rightarrow$ may be strongly affected

Example Output

Gene	Control	Treated	Fold Change	logFC
BRCA1	5.5	12.5	2.27	+
TP53	8.5	15.5	1.82	+
EGFR	10.5	18.5	1.76	+
MYC	7.5	21	2.8	highest